Execution Governor Limits in Salesforce CRM :

Controlling resource consumption during Apex execution

Governor Limits = Runtime Restrictions on Salesforce Resources

- Salesforce is a cloud-based CRM platform that uses a multi-tenant architecture, where many organizations share platform resources.
- Governor limits prevent a single transaction from consuming an excessive amount of shared CPU, memory, database, or processing resources.
- Execution governor limits are applied while Apex code, triggers, and other platform processes are running.
- Important limits include SOQL queries, SOSL queries, DML statements, DML records, CPU time, heap size, and callouts.
- When an Apex transaction exceeds a governor limit, Salesforce stops the transaction and raises a runtime exception.
- The purpose of these limits is to maintain platform performance, stability, reliability, and fair resource usage.

Major Execution Governor Limits :

Common limits that developers must consider when designing Apex applications

Limit	Purpose	Why It Matters
SOQL Queries	Controls the number of database queries in one transaction.	Prevents excessive database access.
DML Statements	Controls insert, update, delete, undelete and other DML operations.	Encourages efficient bulk processing.
DML Rows	Limits the total number of records affected by DML.	Prevents very large transactions.
CPU Time	Controls processor time consumed by Apex and related execution.	Protects shared computing resources.
Heap Size	Controls memory used for objects, collections, and data during execution.	Prevents excessive memory consumption.
Callouts	Limits external HTTP/web-service requests during a transaction.	Helps control integration resources.

• *Limits vary by execution context; synchronous and asynchronous Apex can have different resource limits.*

How Execution Governor Limits Work :

Explanation on the left and a simple execution flow on the right

LEFT — Explanation

1. A user action, trigger, API request, or automation starts an Apex transaction.

2. Salesforce creates an execution context and tracks the resources consumed by that transaction.

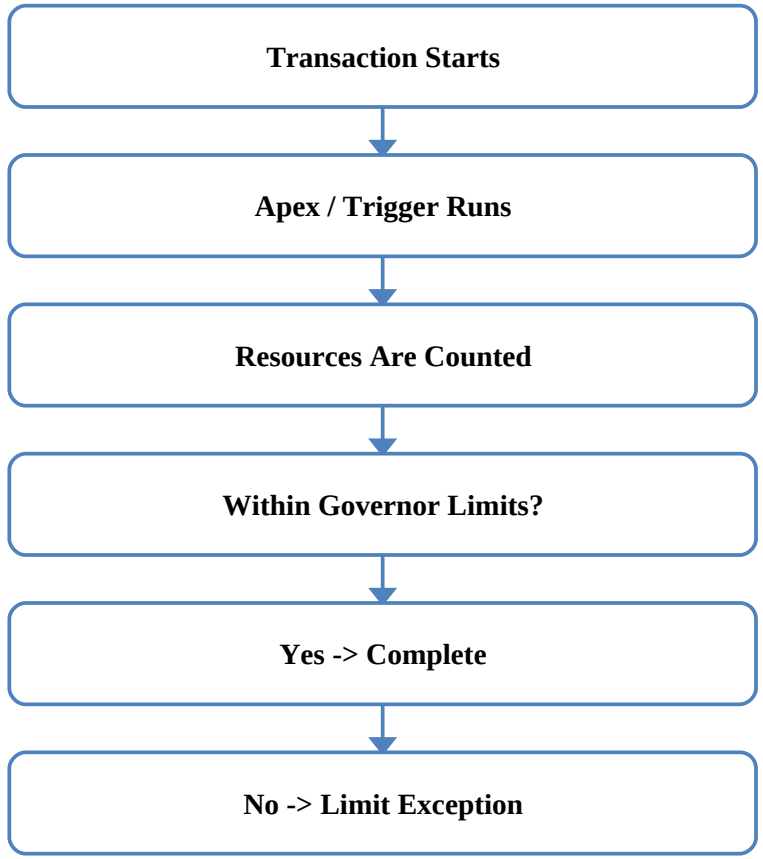
3. Every SOQL query, DML operation, CPU operation, and memory allocation counts toward the limits.

4. If the code remains within the limits, execution continues and the transaction can complete successfully.

5. If any governor limit is exceeded, Salesforce throws a runtime exception and the transaction fails.

6. Developers avoid these failures by bulkifying code, reducing queries and DML, and using asynchronous Apex.

RIGHT — Simple Flow Chart



Best Practices, Applications and Conclusion :

Writing Apex that works efficiently within Salesforce execution limits

Best Practices

- Use bulkified Apex to process collections instead of single records.
- Avoid SOQL queries and DML statements inside loops.
- Query only the fields and records that are actually required.
- Use Lists, Sets, and Maps to organize and reuse data efficiently.
- Use asynchronous Apex for suitable long-running or large-volume operations.

Applications

- Salesforce triggers and automated business processes.
- Customer, sales, service, and account data processing.
- Large-volume data operations and scheduled jobs.
- Integration with external systems through callouts.
- Custom CRM applications developed using Apex.

Key Takeaways

- Governor limits protect the shared Salesforce environment.
- Limit exceptions usually indicate inefficient resource usage.
- Understanding limits helps developers design scalable applications.
- Efficient coding improves performance and reliability.
- Governor limits are a design consideration, not simply a restriction.

Conclusion: Understanding and respecting execution governor limits is essential for secure, scalable, reliable, and high-performing Salesforce CRM applications.